

Java Core Interview Preparation

Purpose

This document is a running reference guide for Java Core concepts covered during the interview preparation sessions.

The goal is not only to know syntax, but to:

- Understand runtime behavior
 - Explain concepts clearly in interviews
 - Understand trade-offs and edge cases
 - Build strong backend engineering fundamentals
-

1. Primitive vs Reference Types

Primitive Types

Primitive types store actual values.

Examples:

```
int x = 10;  
boolean flag = true;
```

Characteristics:

- Store raw values directly
 - Fixed size
 - Faster access
 - Not objects
 - Stored as local variables in stack frames
-

Reference Types

Reference variables store references to objects.

Example:

```
Person p = new Person("John");
```

Characteristics:

- Reference variable points to an object
 - Object itself lives in heap memory
 - Multiple references can point to the same object
-

Pass-by-Value in Java

Java is ALWAYS pass-by-value.

Primitive Example

```
static void change(int x) {  
    x = 20;  
}
```

A copy of the primitive value is passed. Original value remains unchanged.

Object Example

```
static void change(Person p) {  
    p.name = "Doe";  
}
```

A copy of the reference is passed. Both references point to the same object. Mutation affects the shared object.

Reassigning a Reference

```
static void change(Person p) {  
    p = new Person("Doe");  
}
```

This does NOT affect the original reference. Only the local copy changes.

Important Distinction

Mutating Object

```
p.name = "Doe";
```

Affects all references pointing to the object.

Reassigning Reference

```
p = new Person("Doe");
```

Only affects the local reference variable.

2. Stack vs Heap Memory

Stack Memory

Stores:

- Method stack frames
- Local variables
- Primitive local variables
- Reference variables

Characteristics:

- Fast
- Automatically cleaned when method exits
- Thread-specific

Example:

```
Person p = new Person("John");
```

`p` lives in stack.

Heap Memory

Stores:

- Objects
- Instance fields
- Arrays

Characteristics:

- Shared across threads
- Managed by Garbage Collector
- Slower than stack access

Example:

```
new Person("John")
```

The actual object lives in heap.

Example Memory Model

```
Person p1 = new Person("A");  
Person p2 = p1;
```

Stack:

- p1 -> reference
- p2 -> same reference

Heap:

- One Person object

Mutating through either reference affects the same object.

Reachability and Garbage Collection

Objects remain in memory as long as at least one reachable reference points to them.

Example:

```
String s = "abc";  
s = s + "d";
```

The original "abc" object still exists. If no references point to it anymore, it becomes eligible for garbage collection.

3. String Internals

String Pool

Java stores string literals in a special memory area called the String Pool.

Example:

```
String a = "hello";  
String b = "hello";
```

Both references point to the same pooled object.

Thus:

```
a == b
```

returns:

```
true
```

== vs **equals()**

==

Compares references.

equals()

Compares actual content.

Example:

```
String a = "hello";  
String b = new String("hello");
```

```
a == b
```

returns:

```
false
```

because references differ.

```
a.equals(b)
```

returns:

```
true
```

because content is identical.

4. Compile-Time vs Runtime String Concatenation

Compile-Time Concatenation

```
String a = "he" + "llo";  
String b = "hello";
```

Compiler optimizes this into:

```
String a = "hello";
```

Both references point to the same pooled object.

Result:

```
true
```

Runtime Concatenation

```
String a = "he";  
String b = a + "llo";
```

This happens at runtime. A new String object is created.

Result:

```
b == "hello"
```

returns:

```
false
```

Final Variables and Compile-Time Constants

```
final String a = "he";  
String b = a + "llo";
```

Because `a` is a compile-time constant, concatenation is optimized during compilation.

Result:

```
b == "hello"
```

returns:

```
true
```

5. String Immutability

What Immutability Means

A String object's internal state cannot change after creation.

Example:

```
String s = "abc";  
s = s + "d";
```

The original object is NOT modified. A new String object is created.

Important Distinction

Immutable Object

The String object itself cannot change.

Mutable Reference

The variable can point to another object.

Why Strings Are Immutable

1. String Pool Safety

Allows safe reuse of pooled strings.

2. Thread Safety

Immutable objects are naturally thread-safe.

3. Security

Critical for:

- file paths
- database URLs
- authentication
- network communication

4. hashCode Caching

Strings are frequently used as HashMap keys. Immutability guarantees stable hashcodes.

6. String Concatenation Performance

Bad Practice

```
String result = "";  
  
for (int i = 0; i < 10000; i++) {  
    result += i;  
}
```

Problems:

- Creates many temporary String objects
 - Repeated content copying
 - Heavy garbage generation
 - Poor performance
-

Internal Transformation

Conceptually transformed into:


```
result = new StringBuilder()  
        .append(result)  
        .append(i)  
        .toString();
```

inside every loop iteration.

Better Approach

```
StringBuilder sb = new StringBuilder();  
  
for (int i = 0; i < 10000; i++) {  
    sb.append(i);  
}  
  
String result = sb.toString();
```

Why StringBuilder Is Better

StringBuilder:

- Uses mutable internal character buffer
 - Reduces allocations
 - Minimizes copying
 - Improves performance significantly
-

StringBuilder vs StringBuffer

StringBuilder

- Not thread-safe
- Faster
- Preferred in most cases

StringBuffer

- Thread-safe
 - Synchronized
 - Slower
-

Default Object Behavior

If a class does not override `equals()`, Java uses the implementation from `Object`:

```
public boolean equals(Object obj) {  
    return this == obj;  
}
```

This means:

- default equality compares references
- NOT object field values

Example:

```
Person p1 = new Person("John");  
Person p2 = new Person("John");  
  
System.out.println(p1.equals(p2));
```

Output:

```
false
```

because the objects are different references.

Value-Based Equality

To compare object state instead of references, override `equals()`.

Example:

```
@Override  
public boolean equals(Object o) {  
    if (this == o) return true;  
    if (!(o instanceof Person)) return false;  
  
    Person p = (Person) o;  
    return name.equals(p.name);  
}
```

hashCode Contract

If:

```
a.equals(b) == true
```

then:

```
a.hashCode() == b.hashCode()
```

MUST also be true.

Failing to maintain this contract breaks hash-based collections.

HashMap Internal Workflow

put(key, value)

1. Compute `hashCode()`
2. Determine bucket index
3. Store entry in bucket

get(key)

1. Compute `hashCode()`
 2. Locate bucket
 3. Use `equals()` to find exact key
-

Important Distinction

hashCode()

Narrows the search area.

equals()

Confirms exact equality.

Hash Collisions

Different objects may:

- produce same hashCode
- map to same bucket

This is called a collision.

Before Java 8:

- collisions stored as linked lists

Java 8+:

- large collision chains become Red-Black Trees

This improves worst-case lookup from:

$O(n)$

to:

$O(\log n)$

Mutable Keys Problem

Using mutable fields in `hashCode()` or comparison logic is dangerous.

Example:

```
Person p = new Person("John");
map.put(p, "Engineer");

p.name = "Doe";
```

Now:

- hashCode changes
- object remains in old bucket
- future lookups fail

Result:

```
map.get(p) // null
```

TreeMap Mutable Key Problem

`TreeMap` depends on ordering/comparison.

If fields used in comparison change:

- tree structure becomes logically inconsistent
 - ordering breaks
 - lookups may fail
-

8. HashMap vs LinkedHashMap vs TreeMap

HashMap

Internal Structure:

- hash table
- buckets

Characteristics:

- no guaranteed ordering
- average $O(1)$ lookup
- fastest general-purpose map

Use Cases:

- general backend logic
 - caching
 - DTO mapping
-

LinkedHashMap

Internal Structure:

- hash table
- linked list preserving insertion order

Characteristics:

- insertion order preserved
- slightly slower than HashMap

Use Cases:

- predictable iteration order
 - API responses
 - LRU cache implementations
-

TreeMap

Internal Structure:

- Red-Black Tree

Characteristics:

- sorted order
- $O(\log n)$ lookup
- keys must be comparable

Use Cases:

- ranking systems
- sorted reports
- range queries

Comparison Table

Type	Ordering	Internal Structure	Lookup
HashMap	None	Hash Table	$O(1)$
LinkedHashMap	Insertion Order	Hash Table + Linked List	$O(1)$
TreeMap	Sorted	Red-Black Tree	$O(\log n)$

9. ArrayList vs LinkedList

ArrayList

Internal Structure:

```
Object[] elements;
```

Characteristics:

- contiguous memory
- fast random access
- cache-friendly
- lower memory overhead

Random access:

```
list.get(index)
```

Complexity: $O(1)$

because index location can be computed directly.

LinkedList

Internal Structure:

```
Node {  
    value  
    next  
    prev  
}
```

Characteristics:

- node-based structure
- poor cache locality
- traversal required for indexed access

Random access:

```
linkedList.get(index)
```

Complexity: $O(n)$

because traversal is required.

Important Misconception

Although node insertion itself is cheap in `LinkedList`, finding the insertion location still requires traversal.

Thus many real-world insertions are still effectively $O(n)$.

Why ArrayList Often Performs Better

Reasons:

- contiguous memory
- CPU cache friendliness

- fewer allocations
 - less pointer chasing
-

ArrayList Resizing

When capacity becomes full:

1. allocate larger array
2. copy old elements
3. insert new element

Worst-case resize: $O(n)$

Amortized $O(1)$

Most insertions do NOT trigger resize.

Occasional expensive resizes are spread across many cheap insertions.

Thus average append complexity becomes:

$O(1)$

This is called amortized $O(1)$.

Why Exponential Growth Is Used

If resizing occurred by only +1 each time:

- copying would happen constantly
- overall insertion cost would degrade toward $O(n^2)$

Exponential growth reduces resize frequency and improves throughput.

Phase 1 Summary — Java Foundations